WEEK 07

像工程師一樣開發：規劃與測試

從這週起，加進專業開發者的做法：先規劃、先寫測試，再讓 AI 一步一步做



影片 01・3:24



影片 02・4:08



作業・2:52



補充 1・3:51



補充 2・2:57

AI Agent Literacy・學習分析工具



本週內容

	影片 01	**軟體工程與測試驅動開發（TDD）**	3:24
	影片 02	**大任務，先規劃：PLANS.md**	4:08
	作業	**規劃並做出「空中之島」**	2:52
	補充 1	**網站是怎麼組成的**	3:51
	補充 2	**Git：版本管理，也是你的安全網**	2:57
	練習	**進入遠端電腦（Mac／Windows）**	–

影片共 17 分 13 秒

從這週起，加進專業開發者的做法

WEEK 1–5

入門版

先體驗 AI Agent
交代任務，看它怎麼做事

把想法做出來
例如 Week 4 的銀杏散步地圖

WEEK 7 起

專業做法

大任務，先規劃
寫成 PLANS.md・影片 02

先寫測試，再寫程式
TDD：紅 → 綠 → 重構・影片 01

每一步都留存檔點
測試全綠就 commit・補充 2

Week 7・影片 01

AI 寫程式這麼快，為什麼還要軟體工程？



AI 寫得越快，越需要軟體工程

什麼是軟體工程

一群人在有限的時間裡，
做出可靠、以後還改得動的軟體的方法。

AI 讓寫程式變快了，難的變成兩件事：

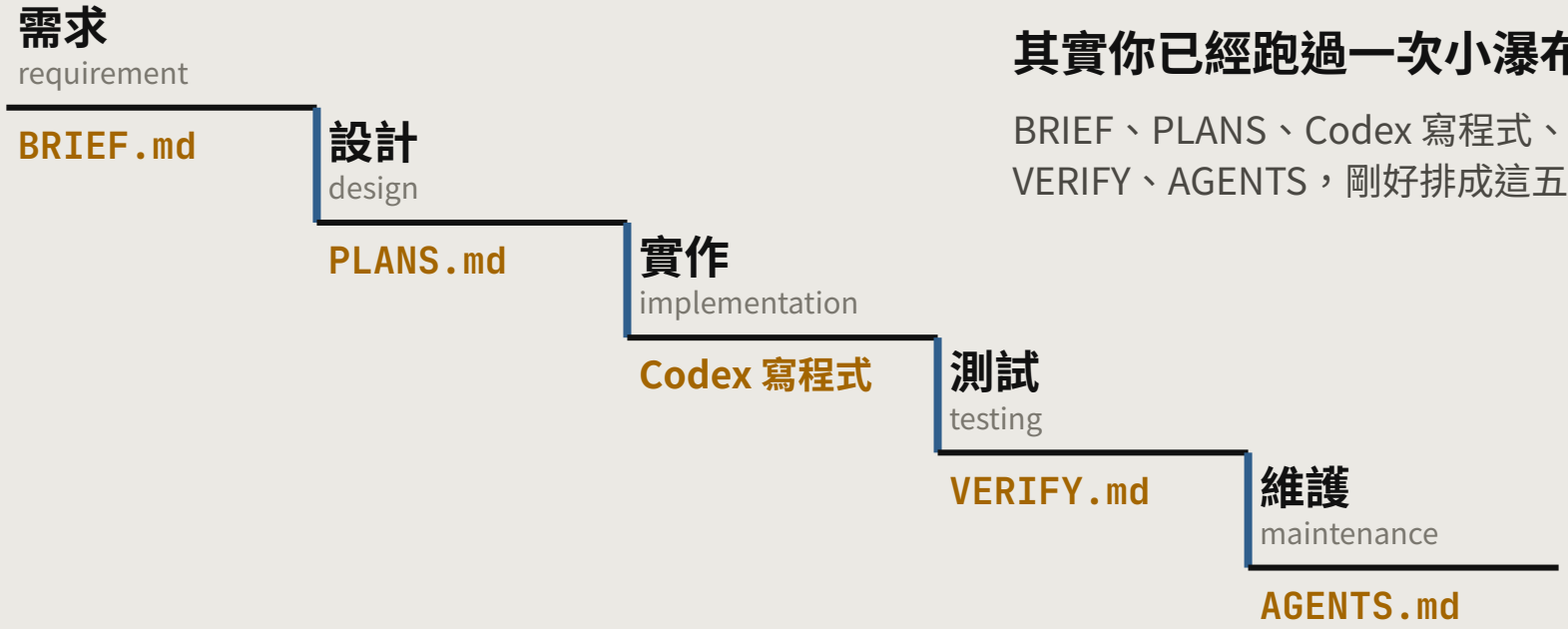
01 **知道要做什麼**

目的地說清楚：BRIEF.md

02 **確定做對了**

寫成測試：VERIFY.md

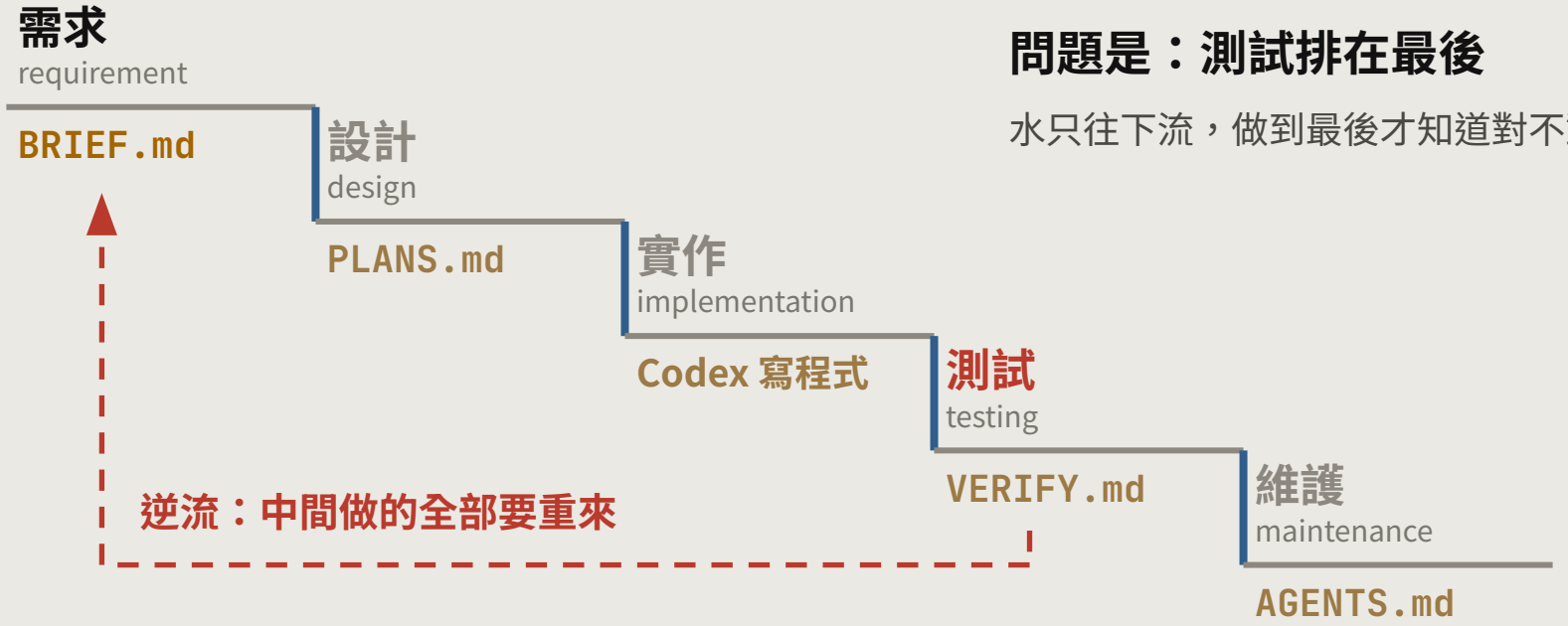
瀑布式開發：一階一階往下流



其實你已經跑過一次小瀑布了

BRIEF、PLANS、Codex 寫程式、
VERIFY、AGENTS，剛好排成這五階。

錯在需求，就得逆流回到最上面



問題是：測試排在最後

水只往下流，做到最後才知道對不對。

只跑一趟瀑布，很冒險

最常被引用的瀑布論文，其實在提醒

"risky and invites failure"

Winston W. Royce，1970

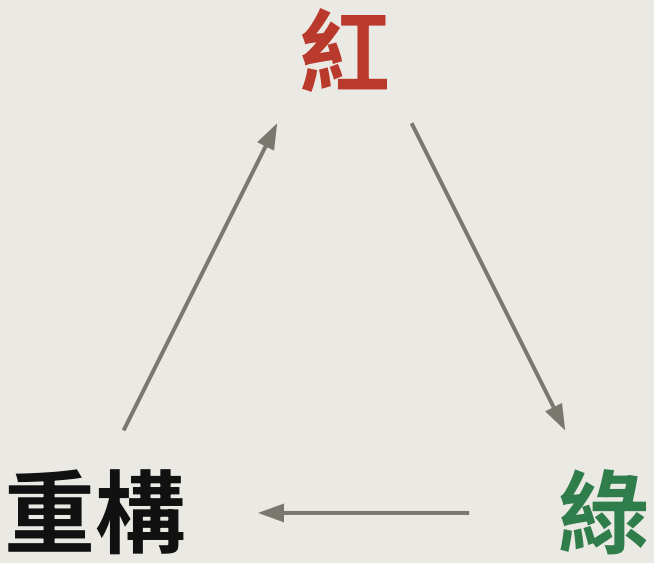
只跑一趟，測試排在最後，問題要到最後才發現。
有趣的是：論文裡沒有出現 waterfall 這個字。

業界什麼時候用瀑布式？

- 流程很熟
- 做過很多次
- 有過去經驗的專案

Source: W. W. Royce (1970). Managing the Development of Large Software Systems. Proceedings of IEEE WESCON.

TDD：把一趟大瀑布，切成很多小瀑布



- **紅**
  先寫一個會失敗的測試
- **綠**
  寫剛好讓它通過的程式
- **重構（refactor）**
  把程式整理乾淨，燈要一直是綠的

測試驅動開發（Test-Driven Development）

Source：Kent Beck，《Test-Driven Development: By Example》，2002。

寫測試：先寫下「給什麼輸入，應該得到什麼」

測試案例（test case）

補交	同一位同學：45 分，補交 67 分	→ **算 67 分**	●
缺考	沒有分數，備註「缺考」	→ **算 0 分**	●
請假	沒有分數，備註「請假」	→ **不列入平均**	●

一條，
就是一個測試案例

寫成程式，
一個指令全部跑完

還沒寫程式，
所以全部是紅燈

影片裡的成績規則接著 Week 6 第 3 關；規則和成績都是示意。

是不是很熟悉？這就是 VERIFY.md

Week 4

Single-turn Prompting

先寫好 BRIEF.md

→

Week 5

VERIFY.md

怎樣算做好

→

Week 7

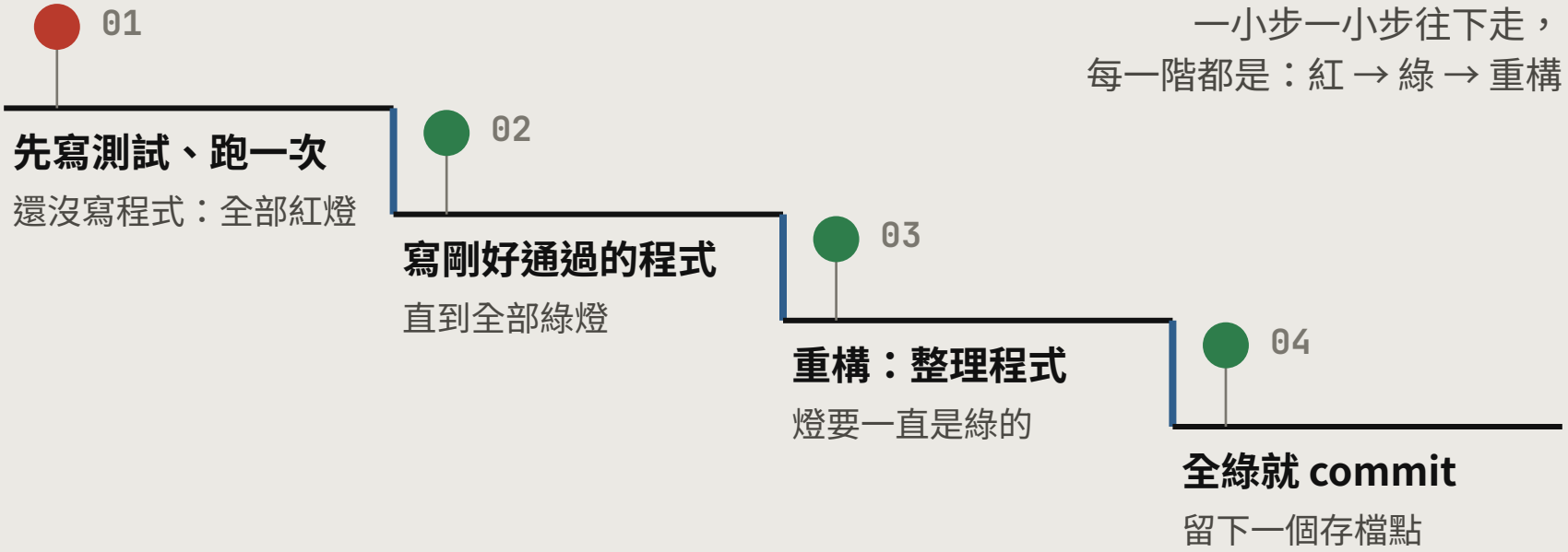
寫測試

先寫測試，再寫程式

都是：先想清楚，結果才有確定性

VERIFY.md 從「可以檢查」再進一步：寫成能自動跑的測試。

在 Codex 裡：先紅燈，再綠燈，再整理



AI 很會說「做好了」
測試不會被說服。

測試每次都全部重跑：哪裡壞了，紅燈馬上告訴你。



回歸（regression）：改了 A，壞了 B

測試清單

	補交	同一位同學：45 分，補交 67 分	→ 算 67 分	●
	缺考	沒有分數，備註「缺考」	→ 算 0 分	●
B	請假	沒有分數，備註「請假」	→ 不列入平均	●
A	補考	原本 40 分，補考 70 分	→ 算 70 分	●

A：新加「補考」的規則

B：原本會過的「請假」變紅了

測試每次都全部重跑，B 一壞，馬上被抓到

小心：AI 可能為了通過，去改測試

想讓紅燈變綠，有兩條路

✓ **把程式改對**
這才是我們要的

✗ **把測試改掉**
燈變綠了，程式還是錯的

AGENTS.md

✓ **不要為了通過而修改測試**

✓ **覺得測試錯了，先問我**

做完用「檢視變更」看：寫程式的那一輪，有沒有動到測試檔。

Week 7・影片 02

大任務，先規劃：PLANS.md



PLANS.md 補上最後一塊：怎麼走到那裡

檔案	回答的問題	誰寫	開始
AGENTS.md	怎麼做事（長期規則）	你寫，Codex 每次都讀	W3
BRIEF.md	做什麼（目的地、限制）	你寫	W4
VERIFY.md	怎樣算做好（你寫的測試）	你寫、你維護	W5
PLANS.md	**怎麼走到那裡** **分幾步、先做什麼、對到測試**	**AI 起草，你審方向**	W7

AI Agent 是透過本地文件和你對話

你的資料夾

`AGENTS.md`	怎麼做事
`BRIEF.md`	做什麼
`VERIFY.md`	怎樣算做好
`PLANS.md`	怎麼走到那裡

Codex 每次工作前都會讀

線上 chat

聊過的話，散在對話裡

換一個對話，就找不到了

本地文件

存在你的硬碟上

換對話、被壓縮了，檔案都還在

檔名全大寫，是大家約好的名字

AGENTS.md
README.md
PLANS.md

大寫：大家約好的名字

看到就知道用途；有的工具還會自動讀，
例如 AGENTS.md、README.md

---

notes.md

小寫：你自己取的名字

自己的工作筆記，名字自己決定。

註：BRIEF.md、VERIFY.md、PLANS.md 是這門課採用的大寫慣例，不是官方標準。

BRIEF 寫做什麼，PLANS 寫怎麼走

	BRIEF.md	PLANS.md
回答	**做什麼**	**怎麼走到那裡**
內容	**目的、限制**	**順序、做法、檔案怎麼放**
多常改	**很少改**	**邊做邊調整**
誰寫	**你**	**AI 起草，你審方向**

判斷方法：**「換一個人來做，這句還成立嗎？」**
成立 → **BRIEF**；可能不同 → **PLANS**

想一想：哪句是 BRIEF、哪句是 PLANS？

1	世界大小固定，32 × 32 × 32 以內	
2	先做能走、能挖、能放，再加材質	
3	遊戲規則和存檔，放在不靠畫面的檔案	

提示：**換一個人來做，這句還成立嗎？**



答案：誰來做都一樣的，才寫進 BRIEF

1	世界大小固定，32 × 32 × 32 以內	BRIEF
2	先做能走、能挖、能放，再加材質	PLANS
3	遊戲規則和存檔，放在不靠畫面的檔案	PLANS



BRIEF　誰來做都一樣（目的、限制），很少改

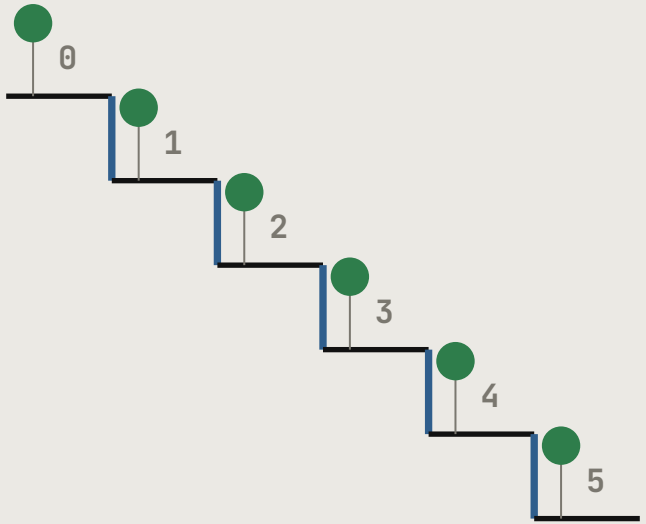
PLANS　換個人可能不同（順序、做法、檔案怎麼放），由 AI 邊做邊調整

PLANS.md：分幾步、先做什麼，每步對到測試

PLANS.md 例子：空中之島

0	測試環境：一行指令跑全部測試	全部
1	小島和走路	V1
2	挖方塊	V2
3	放方塊	V3 V4
4	存檔：關掉再開還在	V5
5	材質和下雨	V6

V1、V2…：VERIFY.md 裡的測試編號



每一步一盞燈：
那一步的測試通過，燈才亮

請 Codex 起草前，先說清楚範圍和驗收

01 有不清楚的，先問我

02 VERIFY.md 每一條都要排進某一步

03 每一步寫出要通過哪幾條測試

04 不做 BRIEF 沒要的，不違反限制

怎麼做、分幾步：讓 AI 決定

這就是你控制規劃方向的方法。

讀 BRIEF.md 和 VERIFY.md，
照這四點起草 PLANS.md。
先不要寫程式。

開始規劃之前，想先問一件事：
存檔要存在瀏覽器裡
（localStorage）嗎？

它先問，你回答；答案也補進 BRIEF.md

Plan mode 的計畫會不見，PLANS.md 不會

Plan mode

- ✓ 先看計畫，再放行寫程式
- ✗ 計畫只存在對話裡
- ✗ 開新對話、被壓縮，就不見了

PLANS.md

- ✓ 存在你的資料夾
- ✓ 換對話、被壓縮，都還在
- ✓ AI 邊做邊勾、邊調整

在 Plan mode 討論出來的計畫，記得請它寫成 PLANS.md 存下來。

你審方向，只問三件事

01 VERIFY 每一條都排進去了嗎？

02 每一步都對到測試嗎？

03 有沒有偷加 BRIEF 沒要的？

AI 的第一版另存成 PLANS.v1.md 對照用。
不對就請 AI 改，你不用自己改寫。

VERIFY.md 是你的，PLANS.md 是 AI 的

你

BRIEF.md	做什麼
VERIFY.md	怎樣算做好
審 PLANS	只審方向，三件事
bug、點子	先寫進 VERIFY

AI

PLANS.md	分幾步、先做什麼
邊做邊調整	回報改了什麼
一次一步	先寫測試再寫程式
做完一步	停下來回報

把規則寫進 AGENTS.md，讓計畫守住方向

AGENTS.md

- ✓ VERIFY.md 不要改，那是我的
- ✓ PLANS.md 你可以自己調整，
  回報時說改了什麼
- ✓ 一次一步：先寫測試、
  確認失敗，再寫程式
- ✓ 不要為了通過而修改測試

之後每一輪只要說

「做 PLANS.md 的下一步」

發現 bug、想到新點子

你先寫進 VERIFY.md，
再請 Codex 寫成測試、確認失敗，
然後修好或做出來。

Week 7・作業

規劃並做出
空中之島

BRIEF · 分批 · PLANS · TDD

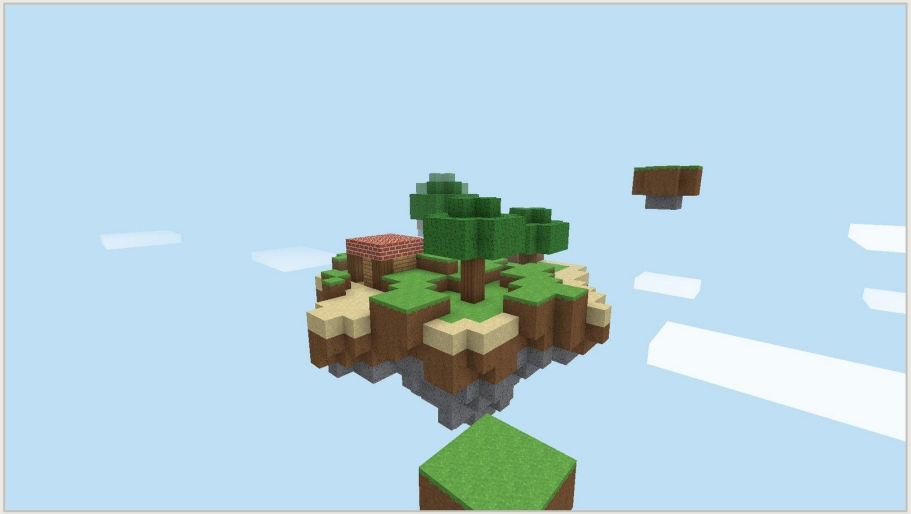
這次的重點：一批一批做，每一批先想好測試







作業：玩老師的空中之島，再自己做出一個



操作：WASD 走、空白鍵跳、按住左鍵挖、
走過去撿、右鍵放回去

照真正的開發方式：分批做

每一批先想好測試，做完、全部通過，
再做下一批。

第 1 批 **挖 → 掉 → 撿 → 放**
測試老師已經想好了

第 2 批 **你自己的點子**
測試由你來想

限制每個人都一樣，直接貼進 BRIEF.md

three.js	30 fps	32×32×32	16×16
靜態網站	**舊電腦也要順**	**世界大小固定**	**只用素材包**
HTML、CSS、JS 不用伺服器、帳號 不用遊戲引擎	學校的舊電腦 內顯或只用 CPU 全螢幕每秒 30 格以上	32 × 32 × 32 格以內 不做無限世界 不做多人	sky-island-assets.zip 方塊材質、圖示 挖方塊的裂痕

這段限制每個人都一樣，請原封不動貼在 BRIEF.md 最後。

作業流程：你先想清楚，Codex 再一步一步做

01	**先玩範例，自己寫 BRIEF.md（不要請 AI 寫）**	你
02	**VERIFY.md：抄第 1 批、寫第 2 批，寫完先 commit**	你
03	請 Codex 挑毛病、補漏掉的	Codex 提，你決定
04	請 Codex 起草 PLANS.md，存 PLANS.v1.md	Codex 起草，你審
05	**AGENTS.md 把規則講好**	你
06	每一輪「做 PLANS.md 的下一步」，一批做完整串玩一遍	Codex 做，你看
07	**交 GitHub repo**	你

第 1 批「挖 → 掉 → 撿 → 放」：老師想好的測試

VERIFY.md　第 1 批・老師想的測試：照抄，不用改

STEP 1 挖方塊

- ☐ 按住左鍵挖 → 越裂越大，不到 1 秒不見
- ☐ **不可以** 只點一下就不見
- ☐ 挖到一半放開 → 裂痕不見，要從頭挖
- ☐ 挖石頭 → 比泥土久，大約 2 秒

STEP 2 掉出來

- ☐ 方塊不見 → 彈出同種類的小方塊，慢慢轉
- ☐ **不可以** 浮在空中、穿過地面

STEP 3 撿起來

- ☐ 走到旁邊 → 飛到身上，那一格數字 +1
- ☐ **不可以** 站在 3 格外也被撿起來

STEP 4 放回去

- ☐ 選那一格按右鍵 → 放出一格，數字 −1
- ☐ **不可以** 數字是 0 還放得出來
- ☐ 整串走一遍 → 數字回到原本

第 2 批：你的點子，測試自己想

01 從 STEP 5 開始，點子大就拆兩步

02 至少 3 條，一條「不可以發生」

03 具體到看得出對錯

04 不要請 AI 想，寫完先 commit

✗ 「下雨要正常」 太模糊

✓ **「按 R → 開始下雨，看得到雨滴往下掉」** 看得出對錯

VERIFY.md 第 2 批

STEP 5 下雨（我的點子）

- ☐ 按 R → 開始下雨，看得到雨滴往下掉
- ☐ 站在屋簷下 → 不會被淋到
- ☐ **不可以** 雨滴穿過屋頂、掉進屋子裡

● 3f2a 第 2 批的測試（自己寫的）

課堂練習：替你的點子寫 3 條測試

3 分鐘

01 **想一個點子**
例如：會下雨、可以種花

02 **寫 2 條：我做了什麼 → 應該看到什麼**
要具體到看得出對錯

03 **再寫 1 條「不可以」**
最容易漏掉的，就是不可以發生的事

04 **檢查**
換一個人來測，看得出對錯嗎？

VERIFY.md 第 2 批

STEP 5 （你的點子）

☐ _____ → _____

☐ _____ → _____

☐ **不可以** _____

請 Codex 挑毛病、補漏掉的，留不留你決定

太模糊、測不出來的
它提建議，你同意才改

漏掉的
補在對應的 STEP 下面，標「（AI 補充）」

島、走路和限制
寫在最前面的 STEP 0「小島」

BRIEF 沒要的
就劃掉

VERIFY.md AI 補充：你決定留哪些

STEP 0 世界不超過 32 × 32 × 32 格
（AI 補充） **留下**

STEP 3 那格滿 64 個 → 撿不起來，留在地上
（AI 補充） **留下**

STEP 4 對自己站的那格按右鍵 → 不會放
（AI 補充） **留下**

STEP 6 ~~肚子會餓，要吃東西~~
（AI 補充） **BRIEF 沒要，劃掉**

想一想：AI 的第一版，哪裡要改？

PLANS.v1.md　AI 的第一版

0	小島和走路	STEP 0
1	挖方塊	STEP 1
2	掉出來、撿起來	STEP 2
3	放回去	（沒寫）
4	做一個合成台	—
5	下雨	STEP 5

對照你的 BRIEF.md 和 VERIFY.md

01 **VERIFY 每一條都排進去了嗎？**

02 **每一步都對到測試嗎？**

03 **有沒有偷加 BRIEF 沒要的？**



三個問題，各抓到一個地方

PLANS.v1.md　AI 的第一版

0	小島和走路	STEP 0	
1	挖方塊	STEP 1	
2	掉出來、撿起來	STEP 2	漏了 STEP 3
3	放回去	（沒寫）	沒對到測試
4	~~做一個合成台~~	—	BRIEF 沒要
5	下雨	STEP 5	

PLANS.md　改好的

0	小島和走路	STEP 0
1	挖方塊	STEP 1
2	掉出來	STEP 2
3	撿起來	STEP 3
4	放回去	STEP 4
5	下雨	STEP 5

不對就請 AI 改，你不用自己改寫；PLANS.v1.md 留著對照。

AGENTS.md：至少寫進這六條

AGENTS.md

- ✓ 不要讀取或複製範例網站的原始碼
- ✓ VERIFY.md 不要改，那是我的
- ✓ PLANS.md 你可以自己調整，回報時說改了什麼，但不能做 BRIEF 沒要的
- ✓ 一次只做一步，做完停下來回報
- ✓ **每一步用 TDD：先把這一步的測試寫好、跑一次確認失敗、commit，再寫程式；全部測試通過才 commit，並在 PLANS.md 勾掉這一步**
- ✓ 不要為了通過而修改測試

每一輪做一步，你看三件事

你說

「做 PLANS.md 的下一步」

- Codex 寫好這一步的測試
- **跑一次，確認失敗 → commit**
- 寫程式
- **全部通過 → commit，勾掉這一步**
- 停下來回報

做完，你看三件事

01 回報
新測試先失敗、最後全部通過

02 「檢視變更」
寫程式的那次，沒有改到測試檔

03 自己玩一下
在 VERIFY.md 把這個 STEP 打勾

一批做完（例如 STEP 4），**自己把整串從頭玩一遍**，都對了再做下一批。

發現 bug：先寫進 VERIFY.md，再修

遠端練習裡的例子

連續挖兩格，第二個撿不起來

01・你

寫進 VERIFY

連續挖兩格 →
兩個都撿得起來

02・Codex

寫成測試

跑一次，
確認失敗（紅）

03・Codex

修好

測試全部通過（綠）

04・你

看三件事

回報、檢視變更、
自己玩一下

想到新點子也一樣：先寫進 VERIFY.md，新點子就是新的一批。

交 GitHub repo：評分最看重你自己想的部分

```
your-repo/
```

BRIEF.md

VERIFY.md

PLANS.v1.md

PLANS.md

AGENTS.md

測試

程式

評分最看重

01 **BRIEF.md**
你自己寫的

02 **第 2 批測試**
要比 PLANS.v1.md 先 commit

03 **維護 VERIFY.md**
後來加的；AI 補的，留下或劃掉

04 **git 紀錄**
每一步都先寫測試、再寫程式

第 2 批做到哪裡都可以，
但做了的每一步都要有測試通過。

Week 7・補充 1

一個網站，是由哪幾塊組成的？

HTML · CSS · JS · 素材

現在看不懂沒關係，等一下一塊一塊拆給你看





網站像一棟房子：HTML、CSS、JS 各管一塊



HTML 隔間

結構和內容・index.html

CSS 裝潢

顏色、排版，還有手機版怎麼排・style.css

JS 水電和開關

按了會怎樣、地圖、讀資料・app.js

素材 圖片和資料

圖片，還有 CSV、JSON 這些資料

拆開自己的網站，就知道每一塊管什麼



完整的網站

四塊都在



拿掉 CSS

內容都還在，
只剩預設的樣子



拿掉 JS

地圖不見了，
按了也沒反應



兩塊都拿掉

剩下 HTML 的
隔間和內容

示範用的是 Week 4 做過的「東京銀杏散步地圖」。

網站就是一個資料夾，瀏覽器把它變成畫面

```
ginkgo-walk/

index.html ← 入口
style.css
app.js
images/
data/spots.csv
```

瀏覽器先拿到 index.html，它再把 CSS、JS、圖片、資料拉進來。上線，就是把這個資料夾放到網路上。

第一堂課裝 Python：讓電腦能執行 Python 程式



做網站不用另外裝：瀏覽器就是網頁的「編譯器」



嚴格說：Python 用的是直譯器；
瀏覽器裡是排版引擎和 JS 引擎。

靜態像印好的菜單，動態像現點現做



Week 6 的考試是靜態網站：進度存在你的瀏覽器，
換一台電腦就不見了。

靜態網站

印好的菜單：
事先做好，每個人拿到一樣的

我們做的網站都是靜態的
也能互動

動態網站

現點現做：
每個人看到的可以不一樣

跨裝置同步、登入、全班共享
需要伺服器和資料庫

懂了網站的組成，才控制得住 AI

01 **指令更準**

例如：「只改 style.css，不要動 HTML 結構」

02 **出問題，知道找哪一塊**

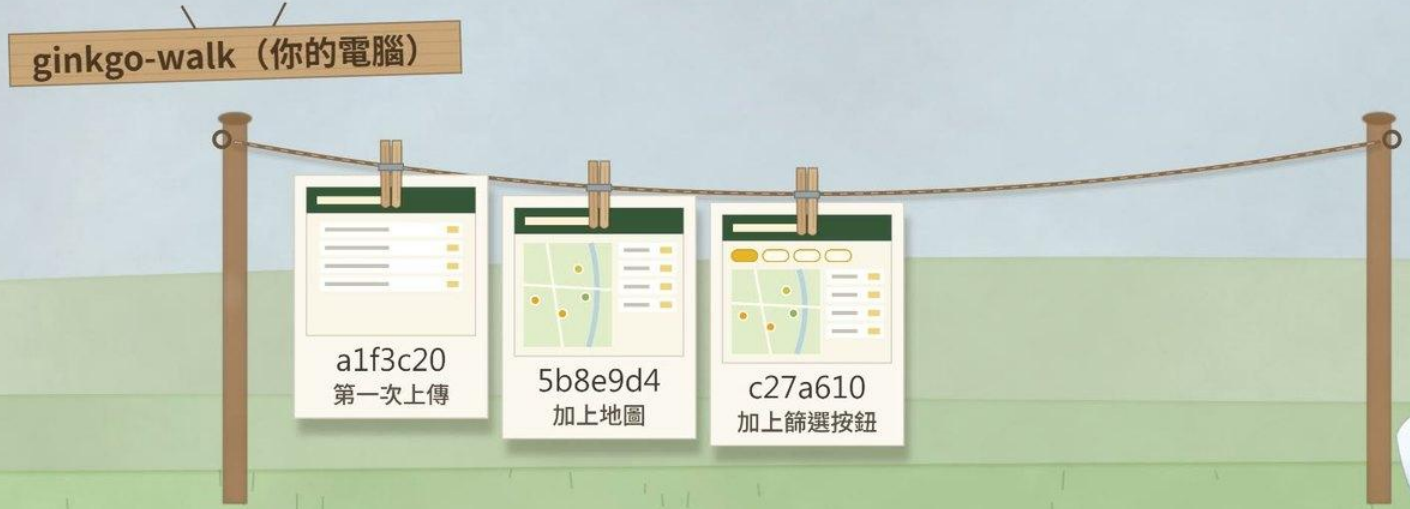
跑版找 CSS；內容寫錯找 HTML 或資料；按了沒反應找 JS

03 **知道能力的邊界**

登入、全班共享資料需要伺服器；要做這些，先跟 AI 說清楚，選對做法

Week 7・補充 2

AI 一次改了好幾個檔案，改壞了怎麼回去？



Git 最重要的是版本管理：commit 就是存檔點

沒有版本管理的時候

報告.docx

報告_final.docx

報告_final_v2.docx

報告_final_v2_改.docx

報告_final_v2_真的最終.docx

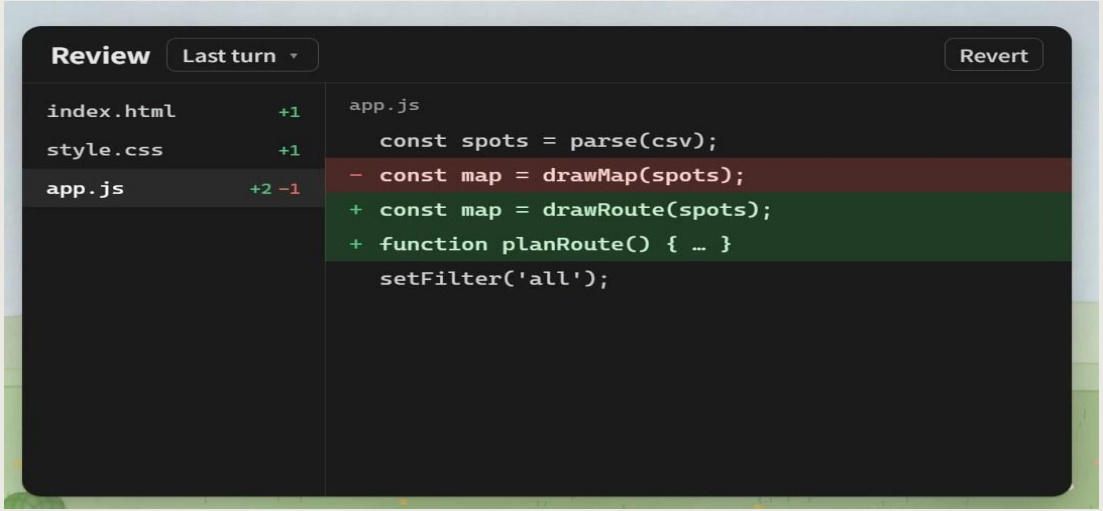
哪一份是最新的？改了什麼？

用 Git：每次 commit 是一個存檔點



可以看歷史、比較差異，回到任何一版。
GitHub 只是把這些版本放到雲端。

AI 一次改好幾個檔案：先看「檢視變更」



示意：Codex app「檢視變更」面板

紅色是刪掉的，
綠色是加上的

一個檔案一個檔案看

選「最近一輪」

只看 AI 這一次改的

專案要是 Git 倉庫

才看得到這些修改

AI 改完，先看改了什麼，再決定要不要 commit。

改壞了很正常，回到上一個存檔點就好

直接跟 AI 說

「剛剛這一輪把地圖弄壞了，
請還原這一輪的修改，回到上一個 commit。」



三個檔案的改動會一起擦掉，網站回到上一個存檔點。

測試全綠才 commit，說明要寫清楚

VERIFY.md 測試案例

- 地圖上有 4 個景點 **通過**
- 按「見頃」，只剩 2 個 **通過**
- 按「規劃散步路線」，畫出路線 **失敗**
- 手機版不跑版 **通過**

有紅燈，就請 AI 修，再跑一次；
全部通過，再 commit。

commit 說明

✗ `update`

之後看歷史，找不到這一版

✓ **地圖加上散步路線規劃，測試全部通過**

寫做了什麼、結果怎樣

練習：在遠端電腦，把作業流程跑一遍



01 先想清楚，再讓 AI 做

BRIEF、VERIFY 先 commit；
Codex 挑毛病、起草 PLANS，你審

02 先紅、再綠、再 commit

每一輪「做 PLANS.md 的下一步」，
自己跑 npm test

03 一批做完，整串玩一遍

發現 bug，先寫進 VERIFY.md

Mac、Windows 都有，每次進入都是全新的電腦。練習裡的 Codex 不會真的寫程式，真正的作業請在你自己的電腦上做。

這週的重點

BRIEF.md	VERIFY.md	PLANS.md	AGENTS.md
做什麼	**怎樣算做好**	**怎麼走到那裡**	**怎麼做事**
你寫	你寫、你維護	AI 起草，你審方向	你寫，Codex 會讀

每一步

紅 → 綠 → 重構 → 全綠就 commit

先想清楚，
再讓 AI 照計畫走

這週要做

看影片 01、02 和兩支補充 → 玩空中之島 → 交 GitHub repo

ai-agent-literacy.pages.dev/weeks/week07/

